

DATATALKSCLUB

MLZOOMCAMP 2025

Week 4 notes

Carlos Leal León

October 20, 2025

General Index

1	Accuracy	4
1.1	Indicator Function	4
1.2	Accuracy and Class Imbalance	4
2	Confusion Table	5
3	Precision and Recall	6
3.1	Precision	6
3.2	Recall	6
4	F1-score	7
5	Receiver Operating Characteristic (ROC) Curves	8
5.0.1	False Positive Rate (FPR)	8
5.0.2	True Positive Rate (TPR)	8
6	Area Under The ROC curve (ROC AUC)	9
7	Cross Validation	10
7.1	Hold-out validation dataset strategy	10
8	Homework 04 Notes	11
8.1	Data preparation	11
8.2	Use ROC AUC to evaluate feature importance	12
8.3	Training the model	12
8.4	Precision and Recall	13
8.5	F1-score	13
8.6	5-Fold CV	14
8.7	Hyperparameter Tuning	14

Figure Index

1	<i>Accuracy.</i>	4
2	<i>Indicator Function.</i>	4
3	<i>Precision.</i>	6
4	<i>Recall.</i>	6
5	<i>F1 Score.</i>	7
6	<i>False Positive Rate.</i>	8
7	<i>True Positive Rate.</i>	8

Equation Index

1	Accuracy	4
3	Indicator Function	4
4	Precision	6
5	Recall	6
6	F1 Score	7
7	False Positive Rate	8
8	True Positive Rate	8

1 Accuracy

Is the number of correct predictions divided by the total number of predictions.

Accuracy [1] is given by:

$$accuracy(y, \hat{y}) = \frac{1}{n_{samples}} \sum_{i=0}^{n_{samples}^{-1}} 1(y_i \hat{=} y_i) \quad (1)$$

Figure 1: *Accuracy*.

Where:

- $\hat{y}_i$: Is the predicted value of the i-th sample.
- y_i : Is the corresponding true value.
- $n_{samples}$: The number of observations.
- $1(x)$: Is the *indicator function*.

1.1 Indicator Function

Is a function that maps elements of the subset to one and all other elements to zero [2].

If A is a subset of some set X then the indicator function of A is:

$$1_A = X \rightarrow \{0, 1\} \quad (2)$$

Defined by:

$$1_A(x) = \begin{cases} 1 & x \in A \\ 0 & x \notin A \end{cases} \quad (3)$$

Figure 2: *Indicator Function*.

1.2 Accuracy and Class Imbalance

Class Imbalance means that there are more instances from one category than the other (in case of a binary classification problem) from two possible categories one of them represents most of the possible outcomes in the available data.

The accuracy is affected by simply predicting the majority class for all instances, failing to identify the minority class (which most of the times is the class of interest). This can lead to a model that appears highly accurate but is not useful to detect the class of interest.

There are acceptable proportions of class imbalance thus it is important to identify how severe is the Class Imbalance in the available data to deal with it properly.

2 Confusion Table

Represents how different types of errors and correct decisions are measured once binary classifiers evaluate observations in a dataset.

In case of a Logistic Regression model you have four different categories:

- True Positive: When a model correctly identifies a positive instance. The actual condition is positive and the prediction result is also positive.
- True Negative: When a model correctly identifies the absence of a condition. The actual condition is negative and the prediction result is also negative.
- False Positive: When a model incorrectly indicates a positive instance when is actually negative. The actual condition is negative but the prediction is positive.
- False Negative: When a model incorrectly indicates a negative instance when is actually positive. The actual condition is positive but the prediction is negative.

Now in terms of the Confusion table categories accuracy is: The sum of True Negatives and True Positives divided by the total of observations.

3 Precision and Recall

3.1 Precision

Precision tells you the proportion of True Positive predictions out of all instances predicted as positive. In other words the fraction of positive predictions that are correct. Focuses on correctness of the positive predictions.

From the **predicted** positives, How many were predicted right?

You can calculate it with the following formula:

$$Precision(P) = \frac{TP}{TP + FP} \quad (4)$$

Figure 3: *Precision*.

Where:

- TP: From the Confusion Table, represents the True Positive category.
- FP: From the Confusion Table, represents the False Positive category.

3.2 Recall

Recall measures the ability of a model to identify all relevant instances within a dataset. Specifically the proportion of actual positive cases that are correctly predicted as positive. In other words the fraction of correctly identified positive instances. Focuses on completeness how well the model finds all relevant cases.

From the **real** positives, How many were predicted right?

You can calculate it with the following formula:

$$Recall(R) = \frac{TP}{TP + FN} \quad (5)$$

Figure 4: *Recall*.

Where:

- TP: From the Confusion Table, represents the True Positive category.
- FN: From the Confusion Table, represents the False Negative category.

4 F1-score

As Precision and Recall are conflicting, i.e. when one grows, the other goes down. They are often combined into the F1-score a metric that takes into account both.

The formula for computing F1 is given by:

$$F1 = 2 \cdot \frac{(P \cdot R)}{(P + R)} \quad (6)$$

Figure 5: *F1 Score*.

Where:

- P: Is Precision.
- R: Is Recall.

5 Receiver Operating Characteristic (ROC) Curves

Is a graphical plot that illustrates the diagnostic ability of a binary classifier system as its discrimination threshold is varied. The plot is defined as the True Positive Rate against the False Positive Rate.

The curve visualizes the trade-off between sensitivity and specificity across all possible classification thresholds with the diagonal line representing a random classifier with no discriminative ability.

5.0.1 False Positive Rate (FPR)

The fraction of False Positives divided by the total number of negatives, False Positives and True Negatives.

You can calculate it with the following formula:

$$FPR = \frac{FP}{TN + FP} \quad (7)$$

Figure 6: *False Positive Rate.*

5.0.2 True Positive Rate (TPR)

The fraction of True Positives divided by the total number of positives, False Negatives and True Positives.

You can calculate it with the following formula:

$$TPR = \frac{TP}{TP + FN} \quad (8)$$

Figure 7: *True Positive Rate.*

ROC curves need to be compared against a point of reference to evaluate its performance, so the corresponding curves of random and ideal models are required.

You can use `roc_curve` [3] from scikit-learn to obtain the rates and thresholds.

However, AUC may not be reliable for highly imbalanced datasets, where precision-recall curves (PRCs) and the area under those curves (AUC-PR) are often recommended as better alternatives for evaluating model performance.

6 Area Under The ROC curve (ROC AUC)

Represents the probability that the model, if given a randomly chosen positive and negative example will rank the positive higher than the negative [4].

AUC is a useful measure for comparing the performance of two different models, as long as the dataset is roughly balanced. The model with greater area under the curve is generally the better one.

The points on ROC curve closest to (0, 1) represent a range of the best-performing thresholds for the given model. The threshold you choose depends on which metric is most important for the specific use case.

If False Positives (false alarms) are costly it may be better to choose a threshold with lower FPR.

If False Negatives (missed True Positives) are costly it may be better to choose a threshold with higher TPR.

7 Cross Validation

Refers to evaluating the same model on different subsets of a dataset, getting the average prediction, and spread within predictions. This method is used in the parameter tuning process where the best parameters are identified and selected.

- Divide the training dataset into k partitions
- Train a model with $k-1$ partitions of the train dataset
- Evaluate the trained model on the remaining subsets or partitions
- Calculate the average evaluation metric for all the folds

When a dataset is large you should use the hold-out validation dataset strategy.

When dataset is small or you want to know the standard deviation of the model accross different folds you can use the cross-validation approach.

Scikit-learn provides the Kfold from the sklearn model selection class for calculating the cross validation with k folds [5].

7.1 Hold-out validation dataset strategy

This method divides the data into a training set and a test (or hold-out) set, with common split ratios being 70:30, 80:20, or 60:40, depending on the dataset size. The training set is used to train the model, while the hold-out test set provides an unbiased assessment of the model's performance on unseen data. This separation ensures that the evaluation reflects how well the model generalizes to new, previously unknown data.

In more advanced applications, the hold-out strategy can involve three distinct datasets: a training set, a validation set, and a test (hold-out) set. The validation set is used during the training process to monitor model performance and tune hyperparameters, helping to prevent overfitting by selecting the model with the best performance on this unseen data. Since hyperparameter tuning can inadvertently overfit to the validation set, a separate test set referred to as the hold-out set—is used only once, after final model selection, to provide a reliable estimate of the model's real-world performance.

8 Homework 04 Notes

I found difficult in this homework to justify my results against the available answers as there are differences that I was not able to reduce without following other steps which are not mentioned in the instructions. And in other cases such alternatives were not enough and seem to be far from the suggested possible answers.

You can find the lecture notebook here [6].

And my homework notebook here [7]

8.1 Data preparation

Steps here were common at this point.

- Handle missing values
- Split dataset into train, validation and test splits.

I noticed that my findings changed a bit by how the splits are made.

- Test size of 0.4: Using this test size yields a 60:40 split for train and val_test. Later using a test size of 0.5 on the val_test split yields 20:20 partitions for the validation and test splits.
- Test size of 0.2: Using this test size yields a 80:20 split for train_f and test. Later using a test size of 0.25 on the train_f split yields 60:20 partitions for the train and validation splits.

Even though the partitions are of the same size, the way they are selected is random as expected but this in later steps shows a small difference in your findings.

Another important thing that should be taken into account is the Class Imbalance. In this case the dataset does not show a severe Class Imbalance so you must be fine using it as is. I did try using the stratify parameter when splitting the dataset but it made no difference to come close to the available answers. As you could tell the Class Imbaance is not as bad so to me it made sense that it was not significant at all.

8.2 Use ROC AUC to evaluate feature importance

The importance of numerical features can be approached by using the ROC AUC. Each of the numerical features was used as as if it was a prediction and then using the `roc_auc_score(y_true, feature)`. From the instructions it can be misleading as it mentions to compute the AUC. In my eyes this is more a correlation than feature importance. As later in the instructions it is mentioned that you can end up with a negative result and you can change the direction of the correlation. When you are about to calculate the `roc_auc_score` you have to take into account a parameter called `multi_class`, as it raises an exception if not specified.

`multi_class={'raise','ovr','ovo'}, default='raise'`. Only used for multiclass targets. Determines the type of configuration to use. The default value raises an error, so either 'ovr' or 'ovo' must be passed explicitly.

'ovr': Stands for One-vs-rest.

Computes the AUC of each class against the rest.

This treats the multiclass case in the same way as the multilabel case.

Sensitive to class imbalance even when `average == 'macro'`, because class imbalance affects the composition of each of the 'rest' groupings.

'ovo': Stands for One-vs-one.

Computes the average AUC of all possible pairwise combinations of classes.

Insensitive to class imbalance when `average == 'macro'`.

Binary classification involves assigning each instance to one of two classes, such as spam or not spam.

In contrast, multi-class classification involves assigning each instance to one of more than two classes, such as identifying a fruit as an apple, orange, or banana.

8.3 Training the model

You are instructed to train a model using:

- `solver`: liblinear
- `C`: Set to 1.0
- `max_iter`: Set to 1000

One thing to note here is that as the solver is set to liblinear, one should use a value for the `random_state` parameter as when it is not specified a random number instance is created internally thus leading to different results each time the model is trained on same data. Other approach could be using a different solver which does not require the `random_state` parameter but it may require other parameter values or extra pre-processing steps.

So from here I started to notice that my results were a bit off from the proposed answers and when you start looking at the details you start to see important things to keep in mind for reproducibility, of course it is not guaranteed that results are going to be exactly the same across different systems but they can be as close as possible from each other.

In my case I was curious to see if scaling could help and decided to train other model scaling numerical features and compare how the results were affected. You can see the difference from this question and the following questions in the homework notebook [7].

8.4 Precision and Recall

Later you have to calculate Precision and Recall of the trained model, here you have to:

- Evaluate the model on thresholds 0.0-0.1 with steps of 0.01
- For each threshold compute precision and recall
- Plot precision vs recall (line plots) to identify where they intersect

First getting the threshold values, here you get an interesting example on how floating point number precision behaves, I saw that some of the thresholds had values like 0.01010101 or 0.98989899 and I simply adjusted the number of samples required as 0.0 and 1.0 are inclusive.

When calculating precision and recall you face division by 0 so you have to deal with those if calculating them manually.

Now the most confusing part is when you visualize your results and take a look at the intersection between precision and recall to select your answer, I plotted vertical lines of the closest values to the intersection just to see how far they were from it and well the result makes you think that what you are doing is not quite right.

As mentioned I followed the instructions and used scaling to compare my findings, for this question the scaled approach was closer to one of the answers than just by following the instructions.

8.5 F1-score

Now to see how Precision and Recall are conflicting you have to compute the F1-score to take both into account.

Here you can see the concern of the previous answer as the F1-score is calculated by using Precision and Recall.

To get the F1-score you have to compute it for thresholds 0.0-0.1 with steps of 0.01. I simply used the previous findings as I saved them in a Dataframe I had access to all of them.

For this question my answer was looking more like one of the available choices, simply following the instructions. In the other hand the result on the scaled model yield almost the same choice, different yes, but not so far from the same choice.

8.6 5-Fold CV

I noticed something important here, in the lecture notebook the split used for the CV is 80:20, later when the folds are used you can see how each iteration used n-folds train and the rest to validate.

Now you have to evaluate the model on 5 different folds. And use the AUC to evaluate the model on validation data. Then obtain the standard deviation of the scores across different folds to select an answer.

Here it was easier to understand the process using a function to train the model, predict using a different function. It is important to note that when you select the features for the encoding process, you do not include the target variable y .

Here after each fold I calculated the mean and standard deviation as shown in the lecture notebook. Now to select a result rounding to two decimals helped a bit (do not let your eyes fool you).

8.7 Hyperparameter Tuning

Here the same process of 5-Fold CV was used but introducing different values for the Regularization of the Logistic Regression. With each value of the regularization you calculate the mean of the AUC and once all are tested you can select your answer, it is required to round your results to three decimals.

References

- [1] scikit learn. "Accuracy score" [Online], at: accuracy-score [Consulted on October 2025].
- [2] wikipedia. "Indicator function" [Online], at: indicator-function [Consulted on October 2025].
- [3] scikit learn. "roc curves" [Online], at: roc-curves [Consulted on October 2025].
- [4] Developers Google Machine Learning. "ROC and AUC" [Online], at: roc-auc [Consulted on October 2025].
- [5] scikit learn. "KFold" [Online], at: kfold [Consulted on October 2025].
- [6] DataTalksClub (github). "04-evaluation notebook" [Online], at: 04-evaluation [Consulted on October 2025].
- [7] cleall (github). "hw notebook" [Online], at: hw-four [Created on October 2025].